



Strings: Structures and Algorithms

CEN207 Data Structures — Week 12

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027

Today's plan (3 hours)

Hour	Topic
1	C strings, buffers, tries, compressed tries, suffix arrays
2	Naive search, KMP (failure fn + search), Rabin-Karp
3	Boyer-Moore, Z-algorithm, dynamic programming: edit distance, LCS

Learning outcomes: LO.1 (explain data structures) · LO.2 (analyze complexity) · LO.6 (dynamic programming) · LO.7 (choose the right structure)

This week's concepts — where

Concept	Where
C strings, buffers, tries, radix trees, suffix arrays	Sections 1–5
Naive, KMP, Rabin-Karp, Boyer-Moore, Z-algorithm	Sections 6–11
Dynamic programming: edit distance, LCS	Sections 13–14

How the code examples work

- Every idea has a complete **C** and **Java** program
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Sources: `code/week-12/c/` and `code/week-12/java/`
- Each program's expected output is in the week notes

Recap — arrays and the char array (Week 1)

- **Array:** contiguous memory, $O(1)$ access by index
- A C string is nothing but a `char` array
- Week 1's index arithmetic (`base + i`) still applies
- Today adds one convention on top: `'\0'` marks the end

Recap — trees and hashing (Weeks 4 and 6)

- **Trees:** recursive nodes with children (Week 4)
- A trie node is a tree node with one child per letter
- **Hashing:** a function computes a location, $O(1)$ (Week 6)
- Rabin-Karp reuses hashing — but *rolls* it forward

A brand-new idea: dynamic programming

- No prior week broke a problem into **overlapping** sub-problems
- Solve each sub-problem **once**, store it in a table
- Sections 13–14 introduce this from first principles
- Two classic examples: edit distance and LCS

Map of the week — at a glance

Structures	Search	Dynamic programming
C strings, buffers	Naive, KMP, Rabin-Karp	Edit distance
Tries, radix trees	Boyer-Moore, Z-algorithm	LCS
Suffix arrays		

1. C Strings: Memory, `'\0'`, and `strlen`

A question to start

Every structure so far stored its size somewhere. A C string does not. So how does any function know where it ends?

A short history

- C designed at Bell Labs, early 1970s (Dennis Ritchie)
- No length field: just a `char` array + a convention
- `'\0'` (NUL, value 0) marks "the string ends here"
- This single choice enables the buffer overflow

Intuition — a row of mailboxes

- Each mailbox (byte) holds one letter
- No sign says "this is the last occupied box"
- Instead: the first **empty** mailbox marks the end
- Walking past it means walking into someone else's mail

The idea: array + convention

- A C string = a `char` array, nothing more
- `'\0'` (NUL) is the end-of-string marker
- Every operation walks byte by byte to find it
- No shortcut exists — the length is never cached

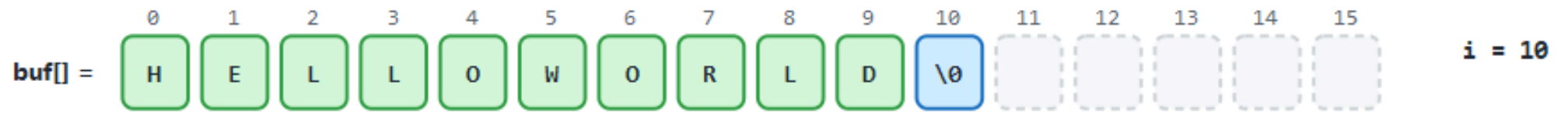
The buffer overflow bug

- `strcpy` -style copying writes until it copies `'\0'` too
- If the destination is smaller than the source: trouble
- Writing past the last valid index is **undefined behavior**
- Today: we always **guard** every write with a bound check

C string memory, step by step

CAP = 16

source text: "HELLOWORLD" (10 letters)

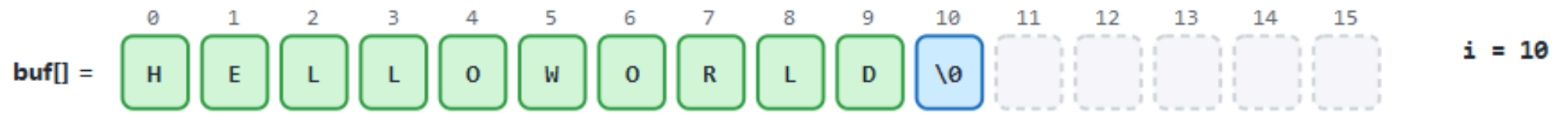


Done: 10 letters + the terminator fit safely in the 16-byte buffer (5 bytes left unused). strlen rescans the string in $O(n)$ every time it is called.

Edge case — overflow, flagged, never executed

CAP = 16

source text: "HELLOWORLD" (10 letters)



Done: 10 letters + the terminator fit safely in the 16-byte buffer (5 bytes left unused). strlen rescans the string in $O(n)$ every time it is called.

Code — the guarded copy loop

```
int i = 0;
while (src[i] != '\0') {
    if (i == cap) break;
    buf[i] = src[i];
    i++;
}
int overflow = (src[i] != '\0');
if (!overflow) buf[i] = '\0';
```

Code — strlen walks it a second time

```
size_t len = 0;  
if (!overflow)  
    while (buf[len] != '\0') len++;
```

- No length is ever cached anywhere
- Calling `strlen` in a loop condition: $O(n)$ becomes $O(n^2)$

Complexity

- Copying a length- L string: $O(L)$
- `strlen`: $O(L)$ — **every single call**, recomputed from scratch
- No caching, no shortcut, ever
- A cached length would break the moment the string changes

Common mistakes

- Forgetting the `+1` for the terminator when sizing a buffer
- Calling `strlen` inside a loop's condition
- Comparing strings with `==` instead of `strcmp`

Mini-quiz

A 10-character word needs a buffer of what size? Why is `char buf[10]` one byte too small?

Answer

11 bytes. Ten characters plus one byte for `'\0'` — forgetting the terminator is this section's classic bug.

2. A Growable String Buffer

A question to start

Section 1's buffer had a fixed capacity,
chosen before one character was written.
What if it could grow itself instead?

Intuition — outgrowing a drawer

- A drawer holds a fixed number of files
- Full drawer: buy a **bigger** one, move every file across
- Buying "one size bigger" every time: constant moving
- Buying **double** the size: moving becomes rare

The idea: double when full

- Track `len` (used), `cap` (allocated), and storage
- `len == cap` : allocate a block **double** the size
- Copy every existing character across, then write the new one
- Doubling makes growth **exponentially rarer** as `n` grows

Growable buffer, step by step

cap = 16

len = 10



Done: 10 letters were appended, 2 growths happened, final capacity = 16 (6 bytes unused). Growth is rare and expensive ($O(\text{len})$) but gives amortized $O(1)$ append overall.

Edge case — smallest possible start

cap = 16

len = 10



Done: 10 letters were appended, 2 growths happened, final capacity = 16 (6 bytes unused). Growth is rare and expensive ($O(\text{len})$) but gives amortized $O(1)$ append overall.

Code — grow, then write

```
void append(char c) {  
    if (len == cap) {  
        cap = cap * 2;  
        buf = realloc(buf, cap);  
    }  
    buf[len] = c;  
    len++;  
}
```

Complexity

- One append: $O(1)$ usually, $O(1 \cdot n)$ on a growth
- n appends total: $O(n)$ — **amortized $O(1)$** per append
- Growth copying forms a geometric series, bounded by n
- Same guarantee as Week 1's dynamic array

Common mistakes

- Growing by a fixed amount instead of doubling: $O(n^2)$ total
- Forgetting to update the pointer after `realloc` (C)
- Allocating `len` bytes when a C string also needs `+1`

Mini-quiz

Why does doubling give amortized $O(1)$,
but growing by a fixed amount of 10
gives $O(n)$ per append on average?

Answer

Doubling: n/k shrinks to $\log n$
growths. A fixed $+10$: $n/10$
growths, each copying up to n bytes.

3. Tries: One Edge Per Character

A question to start

A hash table answers "is X a word?" in $O(1)$. It cannot answer "what words start with X?" at all. What can?

A short history

- Edward Fredkin, "Trie Memory," 1960
- Name comes from "retrieval"
- Usually pronounced "try" to avoid confusion with "tree"
- Still the standard structure behind autocomplete today

Intuition — a signpost tree

- Each fork in the path is labeled with one letter
- Follow a word's letters, one fork at a time
- Shared prefixes share the same early forks
- A small flag marks "a complete word ends here"

The idea: shared prefixes share nodes

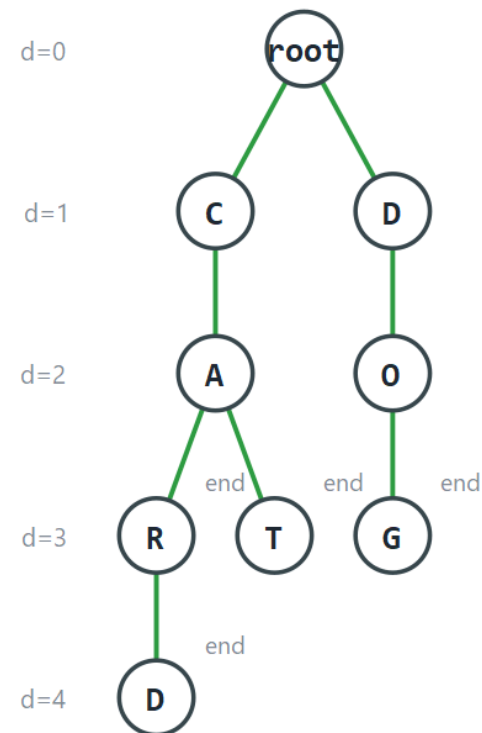
- `insert("CAT")` , then `insert("CAR")` : share `C` → `A`
- `insert("CARD")` : shares `C` → `A` → `R` with `"CAR"`
- `insert("DOG")` : shares nothing, new branch from root
- A node can be **both** "end of CAR" and "path to CARD"

Trie operations

Operation	Cost
insert(word)	$O(L)$, L = word length
search(word)	$O(L)$
prefix check	$O(L)$

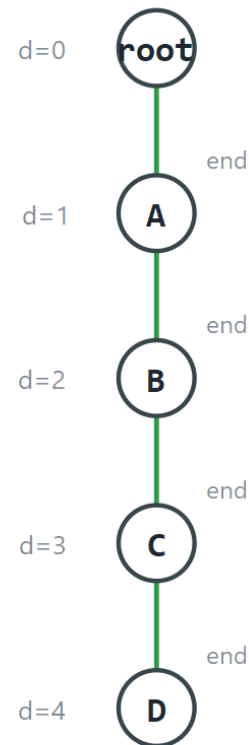
Independent of how many other words are stored!

Trie insert and search, step by step



Done: 4 words were inserted, 3 searches were made (1 found). Every operation costs $O(L)$ in the word's length L -- independent of how many words the trie holds.

Edge case — a branchless chain



Done: 4 words were inserted, 3 searches were made (2 found). Every operation costs $O(L)$ in the word's length L -- independent of how many words the trie holds.

Code — insert

```
void insert(TrieNode *root, const char *word) {
    TrieNode *cur = root;
    for (int i = 0; word[i]; i++) {
        int c = word[i] - 'A';
        if (cur->child[c] == NULL)
            cur->child[c] = new_node();
        cur = cur->child[c];
    }
    cur->isEnd = true;
}
```

Code — search

```
bool search(TrieNode *root, const char *word) {  
    TrieNode *cur = root;  
    for (int i = 0; word[i]; i++) {  
        int c = word[i] - 'A';  
        if (cur->child[c] == NULL) return false;  
        cur = cur->child[c];  
    }  
    return cur->isEnd;  
}
```

Complexity

- Every operation: $O(L)$, L = word's length
- Independent of n , the trie's total word count
- Beats a sorted array or balanced tree: $O(L \log n)$
- Price: memory — many mostly-empty child arrays

Common mistakes

- Confusing "found" with "is a prefix"
- Forgetting a node can be `isEnd` AND have children
- Never freeing the trie in C (a recursive leak, one node at a time)

Mini-quiz

A trie holds "DOG". You call
`search("DO")`. Does it return true
or false — and why?

Answer

False. The path `D → 0` exists, so `"D0"` is a valid **prefix** — but the `0` node is not marked `isEnd`.

4. Compressed Tries (Radix Trees)

A question to start

Inserting "INTERNATIONAL" (13 letters) into a plain trie makes 13 new nodes, each with one child. Can we do better?

A short history

- Donald R. Morrison, "PATRICIA," 1968
- Name: an acronym for a practical retrieval algorithm
- Also called a **radix tree** in general use
- Same idea underlies real-world IP routing tables

The idea: extend, create, or split

- Word matches an edge label completely: **descend**
- No edge starts with the next letter: **new leaf edge**
- Word shares only part of an edge label: **split it**
- A split creates a new branching node at the mismatch

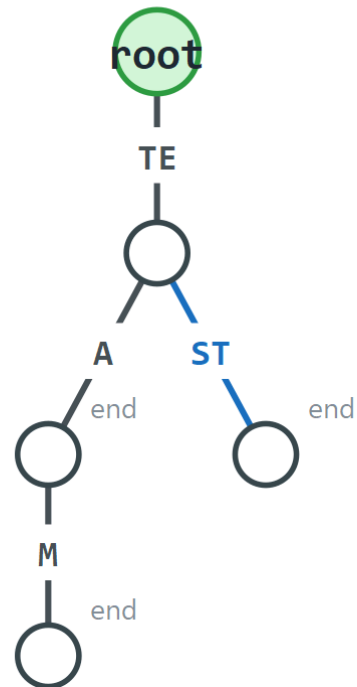
Splitting an edge

- "TEST" alone: one leaf edge, labeled "TEST"
- Insert "TEA": shares only "TE" before diverging
- The "TEST" edge splits at "TE"
- Two children now: "ST" (old) and "A" (new)

Link back — Week 4's Huffman coding

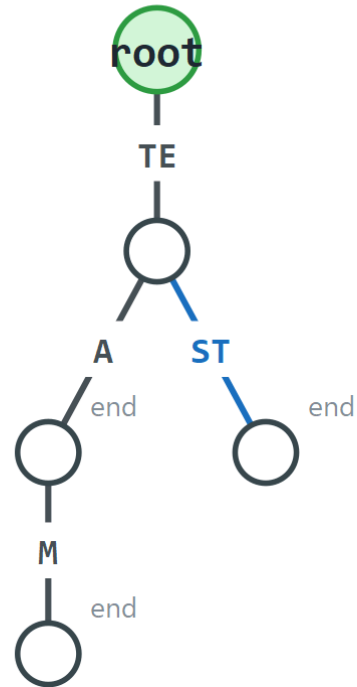
- Huffman: compresses by exploiting **skewed frequencies**
- Compressed trie: compresses by removing **structural** waste
- Huffman shape depends on how OFTEN symbols occur
- Trie shape depends only on the actual characters stored

Compressed trie, step by step



Done: 3 words were inserted, 3 searches were made (1 found). Thanks to the splits, the node count stays far below the words' total letter count.

Edge case — splits nested inside splits



Done: 3 words were inserted, 3 searches were made (1 found). Thanks to the splits, the node count stays far below the words' total letter count.

Code — insert (extend, create, or split)

```
int j = common_prefix_len(word + i, child->label);
if (j == strlen(child->label)) {
    node = child; i += j;    /* descend */
} else {
    RNode *mid = split_edge(node, child, j);
    /* new leaf for the remaining suffix */
}
```

Complexity

- `insert` , `search` : still $O(L)$, exactly like a plain trie
- Savings are in **space**, not time
- Node count bounded by branch points + word endings
- Not by total character count

Common mistakes

- Splitting at the wrong length (must be the common prefix length)
- Forgetting to re-key the split child under its new first letter
- Treating "path exists" as "word found" (same trap as section 3)

Mini-quiz

A compressed trie holds only "APPLE" and "BANANA". How many non-root nodes does it have, and why so few?

Answer

Two. No shared prefix at all, so each word becomes one single leaf edge straight from the root.

5. Suffix Arrays

A question to start

A trie answers "is X a stored word?"

What about "does P occur anywhere inside one long text T?"

The idea: every suffix, sorted

- List **every** suffix of `T`, starting position by position
- Sort those `n` suffixes lexicographically
- A pattern search becomes a **binary search**: $O(m \log n)$
- No sentinel needed — two suffixes always differ in length

Suffix array, step by step

text[] =

0	1	2	3	4	5	6	7	8	9	10
M	I	S	S	I	S	S	I	P	P	I

i=10:

I

i=7:

I	P	P	I
---	---	---	---

i=4:

I	S	S	I	P	P	I
---	---	---	---	---	---	---

i=1:

I	S	S	I	S	S	I	P	P	I
---	---	---	---	---	---	---	---	---	---

i=0:

M	I	S	S	I	S	S	I	P	P	I
---	---	---	---	---	---	---	---	---	---	---

i=9:

P	I
---	---

i=8:

P	P	I
---	---	---

i=6:

S	I	P	P	I
---	---	---	---	---

i=3:

S	I	S	S	I	P	P	I
---	---	---	---	---	---	---	---

i=5:

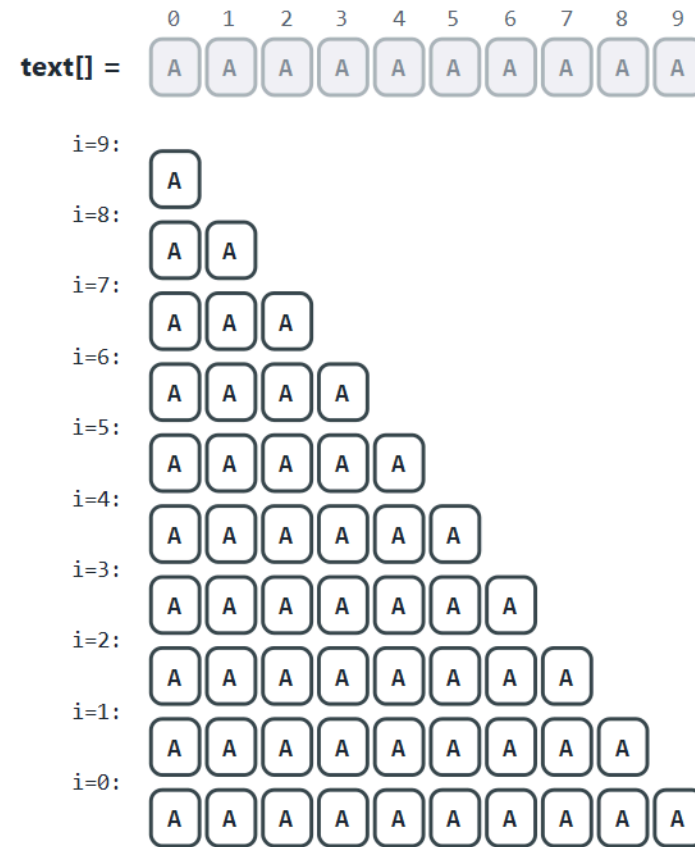
S	S	I	P	P	I
---	---	---	---	---	---

i=2:

S	S	I	S	S	I	P	P	I
---	---	---	---	---	---	---	---	---

Done: the suffix array = [10, 7, 4, 1, 0, 9, 8, 6, 3, 5, 2]. Searching for a pattern can now use binary search over this sorted list -- $O(m \log n)$.

Edge case — every character identical



Done: the suffix array = [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]. Searching for a pattern can now use binary search over this sorted list -- $O(m \log n)$.

Code — insertion sort over suffixes

```
int compare_suffix(const char *text, int a, int b) {  
    return strcmp(text + a, text + b);  
}
```

- Each suffix is a **pointer** into the same buffer
- No character is ever copied during the sort

Complexity

- Insertion sort here: $O(n^2)$ comparisons, worst case
- Each comparison: up to $O(n)$ characters
- Real libraries build it in $O(n \log n)$ or $O(n)$
- Once built: pattern search is $O(m \log n)$

Common mistakes

- Adding an unneeded sentinel character "just in case"
- Comparing suffixes by copying substrings (wastes memory)
- Confusing the array's indices with the suffixes' characters

Mini-quiz

Why can two suffixes of the same text never be byte-for-byte identical?

Answer

They always have different lengths.

Suffixes start at different positions in a finite string, so lengths differ.

6. The String-Matching Problem, and Naive Search

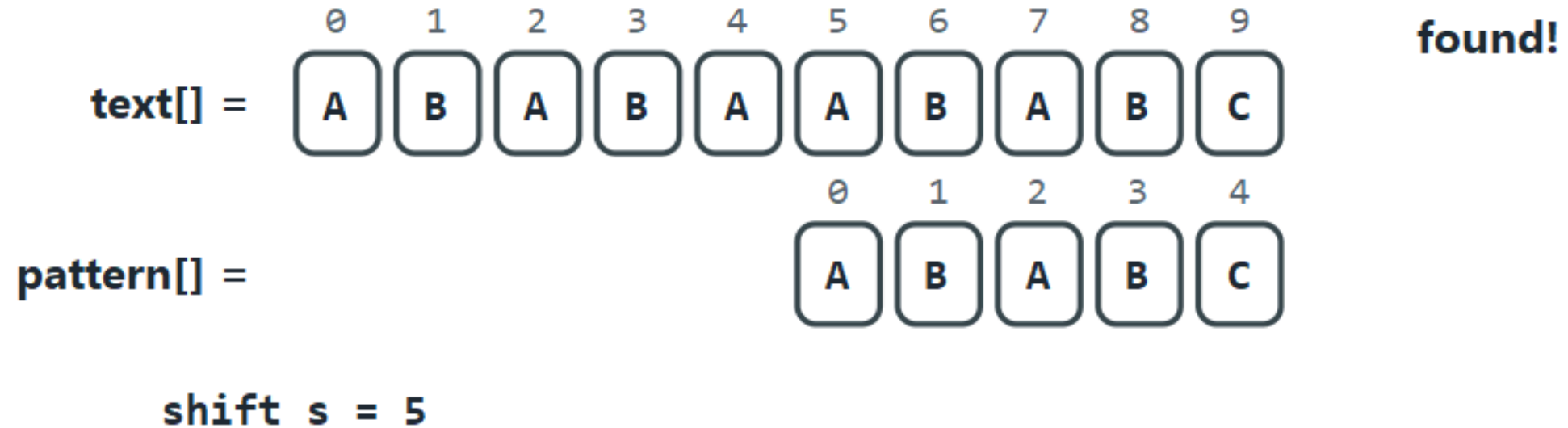
A question to start

Given one text and one pattern, where does the pattern occur? What is the simplest possible way to answer this?

The idea: try every shift

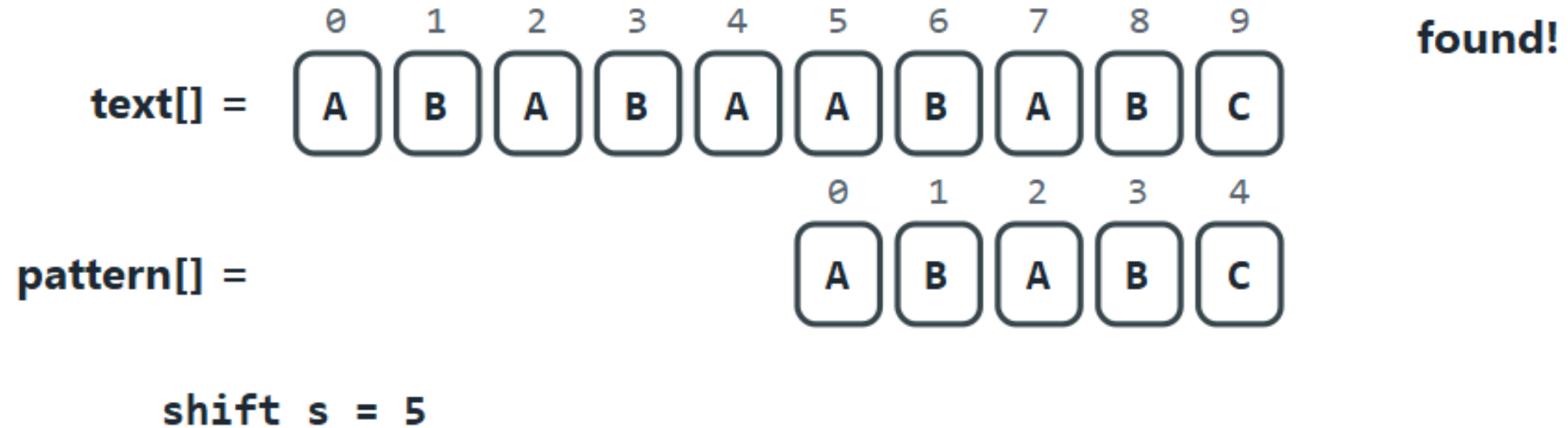
- Slide the pattern across the text, one position at a time
- At each shift: compare left to right
- Stop at the first mismatch, or record a full match
- **Keep going after a match** — occurrences can overlap

Naive search, step by step



Done: 6 shifts were tried, 1 occurrence found (5). Worst case: $O(n*m)$ comparisons.

Edge case — the worst case



Done: 6 shifts were tried, 1 occurrence found (5). Worst case: $O(n*m)$ comparisons.

Code — the double loop

```
for (int s = 0; s <= n - m; s++) {  
    int j = 0;  
    while (j < m && text[s+j] == pattern[j])  
        j++;  
    if (j == m) occ[c++] = s;  
}
```

Complexity

- Best case: $O(n)$ — mismatches happen immediately
- Worst case: $O(n*m)$
- "AAAA...AB" vs "AAAB": nearly every shift, near-full compare
- This worst case is exactly what Sections 7–11 each fix

Common mistakes

- Forgetting matches can overlap (skipping `m` positions after a hit)
- Looping `s` up to `n` instead of `n - m`
- Assuming naive search is always "bad" — it often wins for short input

Mini-quiz

Construct a 2-letter-alphabet text and pattern that forces EVERY comparison to reach the pattern's last character.

Answer

```
text="AAAAAAAAAA", pattern="AAAB" .
```

The first 3 letters always match; only the last, 'B', ever fails.

7. Knuth-Morris-Pratt: the Failure Function

A question to start

Naive search throws away everything it learned after a mismatch. What if the pattern's own structure told us more?

A short history

- Knuth, Morris, and Pratt, 1977 (SIAM J. Computing)
- Independently developed by Morris & Pratt, ~1970
- The table: `lps[ ]` — "longest proper prefix, also a suffix"
- Built by comparing the pattern **to itself**

The idea: $lps[i]$

- For every prefix `pattern[0..i]` ...
- ...the longest proper prefix that is ALSO a suffix
- `"ABAB"` : longest such match is `"AB"` — `$lps[3] = 2$`
- Tells a future search: "this much can be reused"

KMP failure function, step by step

	0	1	2	3	4	5	6	7	8	9
pattern[] =	A	B	A	B	C	A	B	A	B	A
	0	1	2	3	4	5	6	7	8	9
lps[] =	0	0	1	2	0	1	2	3	4	3

Done: $\text{lps[]} = [0, 0, 1, 2, 0, 1, 2, 3, 4, 3]$. This table was built in $O(m)$ and will let KMP search never rewind through the text.

Edge case — the fallback chases a chain

	0	1	2	3	4	5	6	7	8	9
pattern[] =	A	B	A	B	C	A	B	A	B	A
	0	1	2	3	4	5	6	7	8	9
lps[] =	0	0	1	2	0	1	2	3	4	3

Done: `lps[] = [0, 0, 1, 2, 0, 1, 2, 3, 4, 3]`. This table was built in $O(m)$ and will let KMP search never rewind through the text.

Code — building lps[]

```
int len = 0, i = 1;
while (i < m) {
    if (pattern[i] == pattern[len]) {
        lps[i++] = ++len;
    } else if (len != 0) {
        len = lps[len - 1];
    } else {
        lps[i++] = 0;
    }
}
```

Complexity

- $O(m)$ to build the whole table
- `len` decreases at most as many times as it increases
- Total work bounded by $O(m)$, never $O(m^2)$
- Same amortized argument reused in Sections 8 and 11

Common mistakes

- Resetting `len` to 0 on every mismatch (the #1 KMP bug)
- Off-by-one: comparing `pattern[len]`, not `pattern[len+1]`
- Forgetting `lps[0] = 0` is a fixed base case, not computed

Mini-quiz

What does `lps[m-1]` — the very last entry — tell you about the whole pattern?

Answer

The pattern's longest "border" — its longest proper prefix that is also its own suffix, as a whole.

8. Knuth-Morris-Pratt: the Search

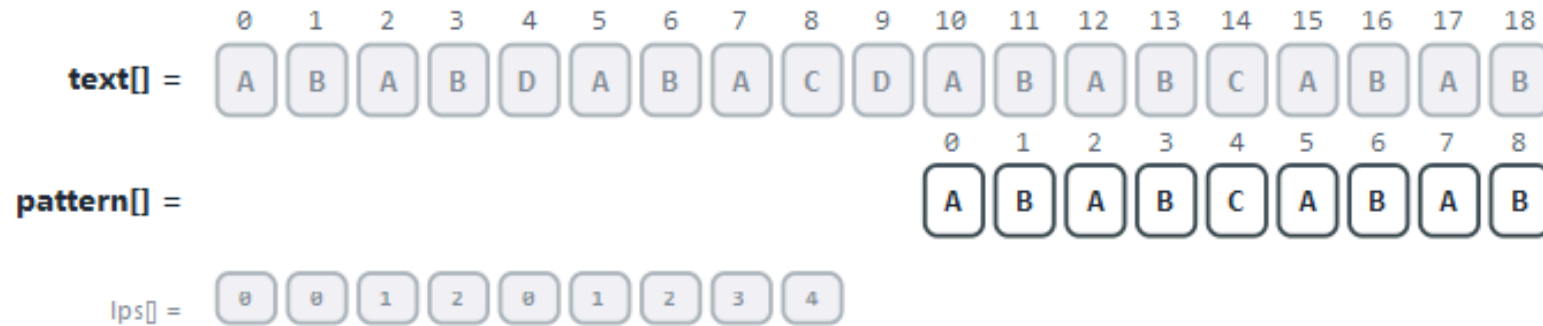
A question to start

With `lps[]` in hand, can we search the text with a pointer that **never moves backward**?

The idea: *i* never rewinds

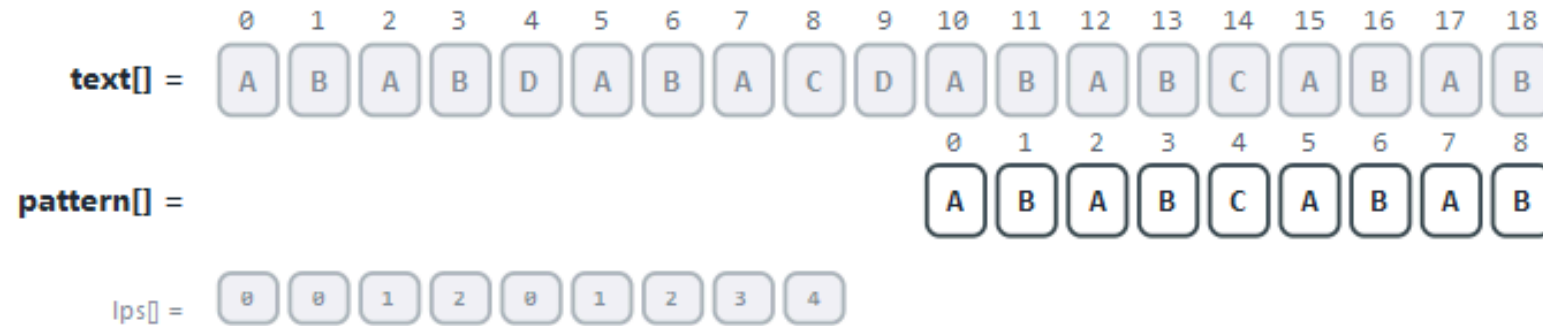
- Two pointers: *i* (text), *j* (pattern)
- Match: both advance
- Mismatch, $j > 0$: *j* falls back to `lps[j-1]` — *i* stays put
- Mismatch, $j == 0$: only *i* advances

KMP search, step by step



Done: *i* advanced exactly 19 times (the text's length), never going back. 1 occurrence found (10). Total time: $O(n + m)$.

Edge case — many lps fallbacks



Done: i advanced exactly 19 times (the text's length), never going back. 1 occurrence found (10). Total time: $O(n + m)$.

Code — the main loop

```
while (i < n) {  
    if (text[i] == pattern[j]) {  
        i++; j++;  
        if (j == m) { occ[c++] = i-m; j = lps[j-1]; }  
    } else if (j > 0) {  
        j = lps[j - 1];  
    } else { i++; }  
}
```

Complexity

- $O(n + m)$: $O(m)$ to build `lps` , $O(n)$ for the search
- `i` advances at most `n` times, total, ever
- Holds for **any** text or pattern — no bad input exists
- Compare to naive search's $O(n*m)$ worst case

Common mistakes

- Advancing `i` on a fallback (defeats the whole guarantee)
- Forgetting to fall back again after recording a match
- Reusing a stale `lps[]` table built for a different pattern

Mini-quiz

Why is $O(n+m)$ achievable for KMP but not for naive search, in one sentence?

Answer

No text character is ever re-examined — naive search's $O(n*m)$ comes exactly from re-examining them.

9. Rabin-Karp: Rolling Hash and Spurious Hits

A question to start

What if, instead of comparing characters at all, we compared a cheap HASH of each window?

A short history

- Rabin and Karp, 1987 (IBM J. Research & Development)
- Reuses Week 6's hashing — but **rolls** it forward
- A hash match is only a **candidate**, never a certainty
- Must always verify before reporting a match

The idea: roll, don't recompute

- Treat each character as a digit in a fixed base
- Sliding the window: remove outgoing char, add incoming
- One $O(1)$ update — the middle characters are never re-read
- A hash **match** must still be **verified** character by character

Rabin-Karp, step by step

base = 31, mod = 101, $H(\text{pattern}) = 60$

	0	1	2	3	4	5	6	7	8	9
text[] =	H	E	L	L	O	W	O	R	L	D
	0	1	2	3	4					
pattern[] =	W	O	R	L	D					

Done: 6 windows were tried, 1 genuine match (5), 0 spurious hits. Average case $O(n + m)$; a worst case with many spurious hits can approach $O(n*m)$.

Edge case — spurious hits

base = 4, mod = 7, $H(\text{pattern}) = 3$

text[] =

0	1	2	3	4	5	6	7	8	9	10
A	A	D	B	D	D	B	C	D	B	B

pattern[] =

0	1	2
A	A	D

Done: 9 windows were tried, 1 genuine match (0), 2 spurious hits. Average case $O(n + m)$; a worst case with many spurious hits can approach $O(n*m)$.

A real bug caught while building this note

- First C draft used `long` for the hash — not `long long`
- On this course's toolchain, `long` is only **32 bits**
- `mod = 1e9+7` overflowed it — silent wrong answer
- Comparing C vs Java output caught it immediately

Code — the rolling update

```
if (s > 0)
    tHash = ((tHash - text[s-1]*hPow % mod + mod)
             * base + text[s+m-1]) % mod;
if (tHash == pHash && strcmp(...) == 0)
    occ[c++] = s;    /* VERIFY, always */
```

Complexity

- Average case: $O(n + m)$ — $O(1)$ per window
- $O(m)$ extra, only for windows whose hash actually matches
- Worst case: $O(n*m)$ if the modulus is small/poorly chosen
- A large prime modulus makes that worst case vanishingly rare

Common mistakes

- Skipping verification (correctness bug, not just speed)
- Using too small a modulus "for simplicity"
- Using a type too narrow for the arithmetic (the `long` bug above)

Mini-quiz

Why must every Rabin-Karp hash match still be verified character by character?

Answer

Pigeonhole principle — more possible substrings than hash values means some **MUST** collide. Not a bug; math.

10. Boyer-Moore: the Bad-Character Rule

A question to start

Every algorithm so far compares left to right. What if comparing RIGHT to LEFT let us jump further?

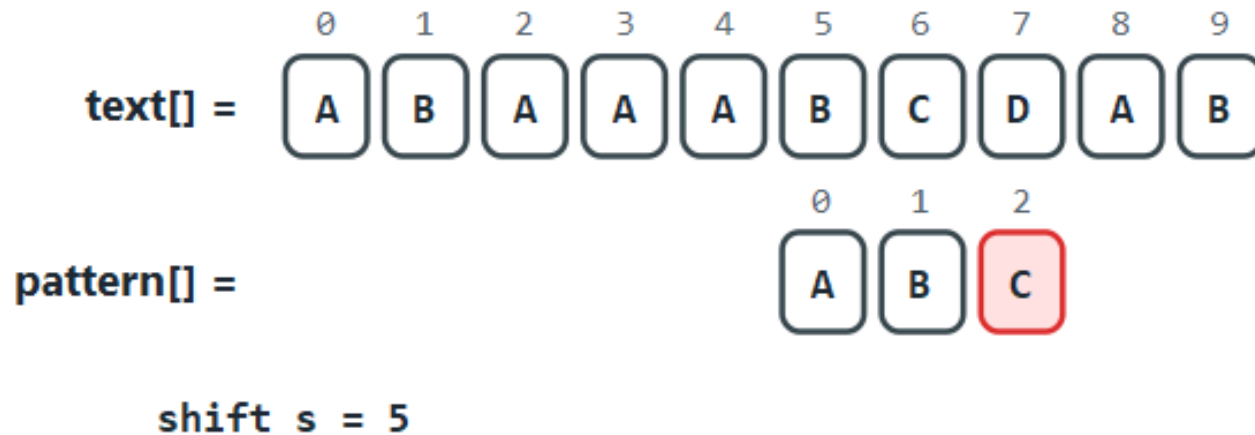
A short history

- Boyer and Moore, 1977 (Communications of the ACM)
- This course covers only the **bad-character** rule
- The full algorithm adds a second "good suffix" rule
- Often the fastest string search in practice today

The idea: scan backward, jump using what you saw

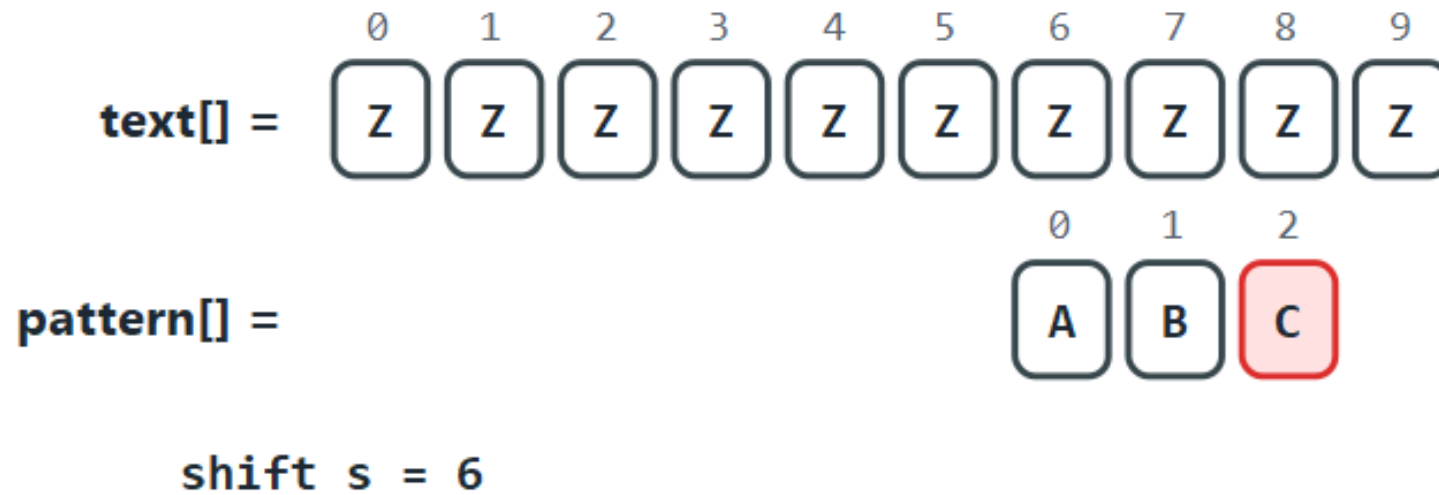
- Compare `pattern[m-1]` first, then `m-2`, ... right to left
- Mismatch: look up that character's LAST occurrence in the pattern
- Character absent from pattern: jump the **whole pattern length**
- Shift never less than 1 (never backward, never stationary)

Boyer-Moore, step by step



Done: 1 occurrence found (4). The bad-character rule gives frequent large jumps on a rich alphabet -- best case $O(n/m)$.

Edge case — the biggest possible jump



Done: 0 occurrences found (none). The bad-character rule gives frequent large jumps on a rich alphabet -- best case $O(n/m)$.

Code — the bad-character table and shift

```
for (int c = 0; c < 256; c++) last[c] = -1;
for (int j = 0; j < m; j++) last[pattern[j]] = j;
/* ... */
int shift = j - last[text[s + j]];
s += shift > 1 ? shift : 1;
```

Complexity

- Best case: $O(n/m)$ — large jumps, rich alphabet
- Worst case: $O(n*m)$ — low-diversity text (bad-character rule alone)
- Often fastest in practice on natural-language text
- The good-suffix rule (not covered) fixes the worst case

Common mistakes

- Forgetting the `shift >= 1` guard (can loop forever without it)
- Building the bad-character table from the TEXT, not the pattern
- Comparing left to right out of habit — loses the whole advantage

Mini-quiz

Why can Boyer-Moore skip text characters naive search is forced to examine?

Answer

One comparison proves several shifts impossible — no match can exist there, so they are never checked.

11. The Z-Algorithm

A question to start

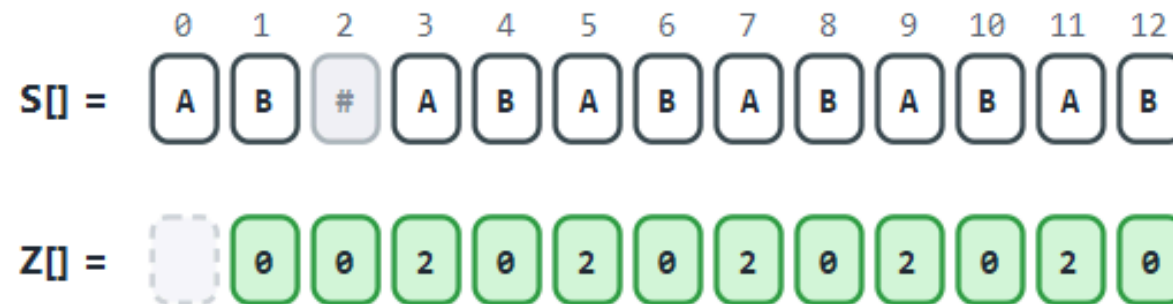
Every algorithm so far built its own bespoke table. Can ONE self-comparison answer the whole search?

The idea: $Z[i]$ and a combined string

- $Z[i]$: how much does $S[i..]$ match S 's own start?
- Let $S = \text{pattern} + \text{'\#'} + \text{text}$ (a separator, unused elsewhere)
- $Z[i] \geq |\text{pattern}|$ in the text part: an occurrence
- A $[1, r)$ window reuses known values — $O(n + m)$ total

Z-algorithm, step by step

pattern="AB" text="ABABABABAB" (combined: $S = \text{pattern} + \# + \text{text}$)



Done: $Z[] = [0, 0, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0]$. Positions in the text part with $Z[i] \geq 2$ are occurrences of pattern: 0, 2, 4, 6, 8. Total time $O(n + m)$.

Edge case — the window is reused constantly

pattern="AAA" text="AAAAAAAAAA" (combined: S = pattern + # + text)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
S[] =	A	A	A	#	A	A	A	A	A	A	A	A	A	A
Z[] =		2	1	0	3	3	3	3	3	3	3	3	2	1

Done: Z[] = [0, 2, 1, 0, 3, 3, 3, 3, 3, 3, 3, 3, 2, 1]. Positions in the text part with Z[i] ≥ 3 are occurrences of pattern: 0, 1, 2, 3, 4, 5, 6, 7. Total time O(n + m).

Code — the Z-array construction

```
if (i < r)
    z[i] = min(r - i, z[i - 1]);
while (i + z[i] < n && s[z[i]] == s[i + z[i]])
    z[i]++;
if (i + z[i] > r) { l = i; r = i + z[i]; }
```

Complexity

- $O(n + m)$ for the whole combined-string Z-array
- Same amortized argument as KMP's lps: window only grows
- $O(1)$ per position to check for an occurrence
- Conceptually the simplest of all five search algorithms

Common mistakes

- Forgetting the separator, or picking one that appears in the text
- Using `Z[i] == m` instead of `Z[i] >= m`
- Treating `Z[0]` as meaningful (it is conventionally unused)

Mini-quiz

How does the Z-algorithm avoid building a SEPARATE table like KMP's lps, in one sentence?

Answer

It doesn't avoid one — `z[]` IS the table, just describing overlap with the WHOLE combined string.

12. Comparing the Five Search Algorithms

Comparison table

Algorithm	Worst case	Typical case	Extra idea
Naive	$O(n \cdot m)$	$O(n)$	Baseline
KMP	$O(n+m)$	$O(n+m)$	<code>lps[]</code> , <code>i</code> never rewinds
Rabin-Karp	$O(n \cdot m)$	$O(n+m)$ avg	Rolling hash, verify
Boyer-Moore	$O(n \cdot m)$	$O(n/m)$	Right-to-left, big jumps
Z-algorithm	$O(n+m)$	$O(n+m)$	One self-comparison array

13. A New Strategy: Dynamic Programming

A question to start

How "different" are two strings? Spell checkers, `git diff`, and DNA tools all ask a version of this question.

Why brute force is too slow

- Recursive definition: match, substitute, delete, or insert
- Correct — but branches into 3 calls at nearly every step
- The SAME sub-problem gets recomputed, exponentially often
- Overlapping sub-problems: the wasted work is fixable

The idea: solve each sub-problem once

- **Optimal substructure:** best answer built from best sub-answers
- **Overlapping sub-problems:** the same one recurs many times
- Solve each sub-problem exactly ONCE, store it in a table
- This is dynamic programming — genuinely new this week

Edit distance: the question

- Fewest single-character edits to turn `a` into `b`
- Insert, delete, or substitute — each costs 1
- Also called **Levenshtein distance** (1965)
- `dp[i][j]` = edit distance between `a`'s first `i`, `b`'s first `j`

The recurrence

- Base case: $dp[i][0] = i$, $dp[0][j] = j$
- Characters match: $dp[i][j] = dp[i-1][j-1]$ (free)
- Otherwise: $1 + \min(\text{diagonal}, \text{up}, \text{left})$
- Fill row by row — every dependency is already computed

Edit distance, step by step

	(empty)	S	I	T	T	I	N	G
(empty)	0	1	2	3	4	5	6	7
K	1	1	2	3	4	5	6	7
I	2	2	1	2	3	4	5	6
T	3	3	2	1	2	3	4	5
T	4	4	3	2	1	2	3	4
E	5	5	4	3	2	2	3	4
N	6	6	5	4	3	3	2	3

ops: substitute match match match substitute match insert

Done: `edit_distance("KITTEN", "SITTING") = 3`. One shortest edit sequence: substitute, match, match, match, substitute, match, insert. The table was filled in $O(n*m)$ time and space.

Edge case — pure insertion

	(empty)	C	A	T	E	R	P	I	L	L	A	R
(empty)	0	1	2	3	4	5	6	7	8	9	10	11
C	1	0	1	2	3	4	5	6	7	8	9	10
A	2	1	0	1	2	3	4	5	6	7	8	9
T	3	2	1	0	1	2	3	4	5	6	7	8

ops: match match match insert insert insert insert insert insert insert insert

Done: $\text{edit_distance}(\text{"CAT"}, \text{"CATERPILLAR"}) = 8$. One shortest edit sequence: match, match, match, insert, insert, insert, insert, insert, insert, insert, insert. The table was filled in $O(n*m)$ time and space.

Code — filling the table

```
if (a[i-1] == b[j-1])
    dp[i][j] = dp[i-1][j-1];
else {
    int best = min(dp[i-1][j-1],
                  min(dp[i-1][j], dp[i][j-1]));
    dp[i][j] = 1 + best;
}
```

Complexity

- $O(n*m)$ time — every cell computed once, in $O(1)$
- $O(n*m)$ space for the full table (needed for traceback)
- Dramatic improvement over the naive recursion's exponential time
- A row-only optimization gets space down to $O(m)$ (length only)

Common mistakes

- Off-by-one: `dp[i][j]` compares `a[i-1]`, not `a[i]`
- Forgetting base cases are `i` and `j`, not zero
- Filling the table in an inconsistent or wrong order

Mini-quiz

Why does dynamic programming need BOTH optimal substructure AND overlapping sub-problems?

Answer

Substructure makes it **CORRECT**.

Overlap makes memoizing **WORTH IT** —
without overlap, nothing is recomputed.

14. Longest Common Subsequence

A question to start

Not "how different" — but "what stayed the SAME, in order, even if not contiguous"?

Subsequence vs substring

- Substring: **contiguous** — "ACE" is NOT a substring of "ABCDE"
- Subsequence: order preserved, gaps allowed
- "ACE" **IS** a subsequence of "ABCDE"
- LCS: the longest sequence common to BOTH strings

The recurrence — almost the same table

- Base case: $dp[i][0] = dp[0][j] = 0$
- Characters match: $dp[i][j] = dp[i-1][j-1] + 1$ (extend)
- Otherwise: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$
- No penalty for a mismatch — only matches ever add anything

LCS, step by step

	(empty)	B	D	C	A	B	A
(empty)	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

LCS so far: BCBA

Done: LCS("ABCBDB", "BDCABA") has length 4, one example: "BCBA". The table was filled in $O(n*m)$ time and space.

Edge case — no shared letters at all

	(empty)	F	G	H	I	J
(empty)	0	0	0	0	0	0
A	0	0	0	0	0	0
B	0	0	0	0	0	0
C	0	0	0	0	0	0
D	0	0	0	0	0	0
E	0	0	0	0	0	0

Done: LCS("ABCDE", "FGHIJ") has length 0, one example: "". The table was filled in $O(n*m)$ time and space.

Code — the recurrence

```
if (a[i-1] == b[j-1])  
    dp[i][j] = dp[i-1][j-1] + 1;  
else  
    dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
```

Complexity

- $O(n*m)$ time and space — identical to edit distance
- Same table shape, filled the same way
- Only the recurrence at each cell differs
- A template you will recognize again in later algorithm design

Common mistakes

- Confusing subsequence with substring (see two slides back)
- Accidentally reusing edit distance's $\min+1$ recurrence
- Reporting only the length when the actual subsequence is wanted

Mini-quiz

Why does LCS use max, while edit distance uses min+1, even though both fill the same table shape?

Answer

Edit distance counts a **COST** (min the cheapest). **LCS** counts a **LENGTH** of matches (max what is already best).

Summary

- **Structures:** char array + `'\0'`, growable buffers, tries, compressed tries, suffix arrays
- **Search:** naive, KMP, Rabin-Karp, Boyer-Moore, Z-algorithm
- **New this week:** dynamic programming — edit distance, LCS
- Same DP table shape, two different recurrences

Exercises and self-check

- 10 exercises in the week notes — trace tables and code by hand
- 10 self-check questions with full worked answers
- Try every "edge" preset in the picker, not just "normal"
- Run every program yourself — outputs are all real, captured output

Looking ahead

- Weeks 13–14: more new material before project weeks 15–16
- The DP table shape (`dp[i][j]` , filled once, read via traceback)
- ...reappears throughout algorithm design, not just for strings
- Keep this week's table shape in mind as a template

References

- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms*
- Knuth, Morris, Pratt (1977) · Boyer, Moore (1977) · Karp, Rabin (1987)
- Fredkin (1960) · Morrison (1968) · Levenshtein (1965)
- Sedgewick & Wayne — *Algorithms*, 4th ed. (Strings chapter)
- Full list with details in the week notes



Questions?

CEN207 Data Structures — Week 12

Next week: continuing new material before Weeks 15–16's project demonstrations